

Authorizing Requests Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-authorization-lab>  <https://github.com/learn-co-curriculum/python-p4-authorization-lab/issues/new>

Learning Goals

- Use the session object to authorize a user to perform actions in an app.

Key Vocab

- **Identity and Access Management (IAM):** a subfield of software engineering that focuses on users, their attributes, their login information, and the resources that they are allowed to access.
- **Authentication:** proving one's identity to an application in order to access protected information; logging in.
- **Authorization:** allowing or disallowing access to resources based on a user's attributes.
- **Session:** the time between a user logging in and logging out of a web application.
- **Cookie:** data from a web application that is stored by the browser. The application can retrieve this data during subsequent sessions.

Introduction

In this lab, we'll continue working on the blog site, and add some features that only logged in users have access to.

There is some starter code in place for a Flask API backend and a React frontend. To get set up, run:

```
$ pipenv install; pipenv shell
$ npm install --prefix client
```

```
$ cd server  
$ flask db upgrade  
$ python seed.py
```

You can work on this lab by running the tests with `pytest -x`. It will also be helpful to see what's happening during the request/response cycle by running the app in the browser. You can run the Flask server with:

```
$ python app.py
```

And you can run React in another terminal from the root project directory with:

```
$ npm start --prefix client
```

You don't have to make any changes to the React code to get this lab working.

Instructions

Now that we've got the basic login feature working, let's reward our logged in users with some bonus content that only users who have logged in will be able to access!

We added a new attribute to our articles, `is_member_only`, to reflect whether the article should only be available to authorized users of the site. We also created two new views: `MemberOnlyIndex` and `MemberOnlyArticle`.

Your goal is to add the following functionality to the new views:

- If a user is not signed in, the `get()` methods in each view should return a status code of 401 unauthorized, along with an error message.
- If the user is signed in, the `get()` methods in each view should return the JSON data for the members-only articles and the members-only article by ID, respectively.

Resources

- [What is Authentication? - auth0](https://auth0.com/intro-to-iam/what-is-authentication)  [_\(https://auth0.com/intro-to-iam/what-is-authentication\)](https://auth0.com/intro-to-iam/what-is-authentication)
- [API - Flask: class flask.session](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)  [_\(https://flask.palletsprojects.com/en/2.2.x/api/#flask.session\)](https://flask.palletsprojects.com/en/2.2.x/api/#flask.session)

This tool needs to be loaded in a new browser window

Load Authorizing Requests Lab (CodeGrade) in a new window